KIM SUNGMIN

김성민

“통과했다”와 “진짜 됐다”를 구분합니다

서버를 만들고, 돌아가게 만들고, 문제가 생기면 찾아냅니다.

CONTACT

brian_als@naver.com

github.com/ks0927



Java · Spring Boot

AWS · Jenkins · Docker

MySQL · RabbitMQ

백엔드와 인프라를 같이 다루는 개발자입니다

교육 프로젝트에서 백엔드 · 프론트엔드 · 인프라를 번갈아 맡으며 서비스 전체 흐름을 익혔습니다.

지금은 SI 스타트업의 공통 인프라를 동료와 함께 운영하고, 의료기기 회사의 대시보드 이전을 혼자 맡고 있습니다.

문제를 먼저 숫자로 재현하고, 고친 뒤 같은 방식으로 다시 측정합니다.

EDUCATION

상명대학교 컴퓨터과학과

2020.03 – 2024.02

학점 3.86 / 4.5

CERTIFICATE

정보처리기사 2023.09

SQLD 2025.04

ADsP 2024.11

TOEIC Speaking IM3 2026.07

AWARD · EDUCATION

SSAFY 13기 특화 프로젝트 우수상

SSAFY 13기 자율 프로젝트 우수상

삼성 청년 SW·AI 아카데미 13기 · 1,620시간

목차

한 줄 결론을 먼저 적었습니다

01	**기술 스택**			4
02	**Addable**	의료기기 대시보드 이전 · 1인	조건을 빠뜨려도 데이터가 새지 않는 구조로	5
03	**Aethra**	공통 인프라 · 배포 자동화 · 2인	초록불이 거짓말을 하고 있었습니다	8
04	**우동**	동아리 운영 · 인프라 + 백엔드	측정이 버그를 가리고 있었습니다	13
05	**HAND**	워치 생체 데이터 · 백엔드	워치는 저장이 끝났는지 알 필요가 없었습니다	16
06	**ETCH**	취업 정보 플랫폼 · 프론트엔드	첫 화면 로딩을 18.9초에서 1.3초로	18
07	**SpootTaxi**	택시 동승 매칭 · 첫 백엔드	서버를 나눴는데 여전히 묶여 있었습니다	20

Backend	Java · Spring Boot · Spring Data JPA	우동 · HAND 백엔드, Addable 1인 개발
	QueryDSL	채팅 · 투표 조회 쿼리 최적화
	RabbitMQ	HAND 생체 데이터 비동기 수집
	Kafka	SpootTaxi 소켓 서버 → API 서버 메시지 전달
	Spring Security · JWT · WebSocket(STOMP)	인증 · 인가, 실시간 채팅
Infra	Docker · Jenkins · Nginx	우동 인프라 전담, Aethra 배포 자동화
	AWS EC2 · RDS · S3	Aethra 공통 인프라 운영, DB 업그레이드
	Prometheus · Grafana · Loki	지표 · 로그 수집, 트래픽 분류
	k6	동시성 · 부하 측정 (전후 비교)
Frontend	React · TypeScript · Vite	ETCH 프론트 담당, Addable 대시보드
DB · 협업	MySQL · MongoDB · Redis	관계형 / 문서형 분리 저장
	Git · GitLab · Jira · Notion · Claude Code	협업 · 문서화

Addable

호흡재활 의료기기 회사의 병원용 대시보드 이전

2026.08 ~ 진행 중

1인 · AethraCorp 고객사 프로젝트

왜 만들었나

Cafe24 호스팅의 PHP 대시보드를 AWS 기반으로 옮기면서

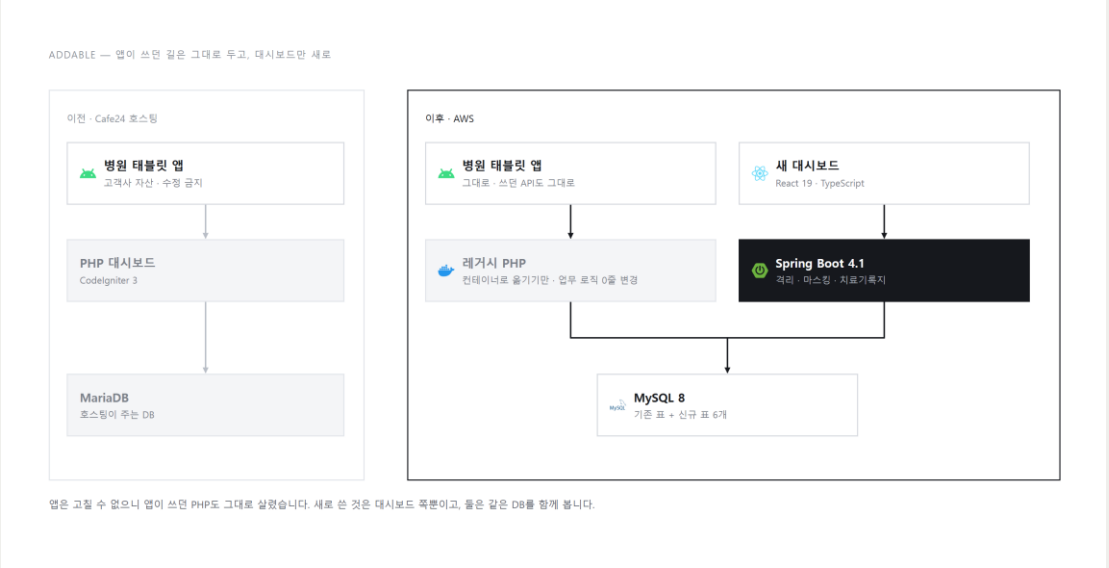
개인정보 보안 요건을 충족해야 했습니다. 병원 태블릿 앱은

수정할 수 없어 기존 API를 그대로 유지해야 했습니다.

주요 기능

환자 · 치료 기록 대시보드 / 병원 간 데이터 격리 / 개인정보 마스킹

치료기록지 PDF 발급 / 그룹 세션 자동 판정



기술 스택

Java 25 · Spring Boot 4.1 · JPA

MySQL 8 · React 19 · TypeScript

Docker · Jenkins · 레거시 PHP 이식

내 기여

1인 수행 (100%)

고객 미팅 · 요구사항 협의 · 기존 서버 조사

DB 이관 설계 · 백엔드/프론트 개발 · 테스트

결과 · 성과

레거시 PHP 업무 로직 0줄 변경

테스트 481개 (실패 0)

이관 전 잘못된 날짜 115,653건 발견

견적 전 조사에서 계약 전제 다섯 개가 달랐습니다

어떻게 했나

개발 전 2주간 기존 서버의 코드와 DB를 읽기 전용으로만 조사했습니다.

확인된 사실과 추정을 구분해 기록하고, 추정은 반드시 코드나

데이터로 검증한 뒤 진행했습니다.

왜 그랬나

전제가 실물과 다르면 범위 분쟁이 납니다.

조사는 개발이 아니라 계약을 지키는 일이었습니다.

계약 · 미팅 전제	실제 확인	결과
병원 · 기기 테이블을 신설한다	이미 존재. 병원 하나가 태블릿 수만큼 중복 등록	“신설” → “통폐합”으로 재정의
모든 API 응답에서 실명 제거	수정 금지인 앱이 실명을 받는 API를 사용	적용 범위를 대시보드 경로로 조정, 합의
MySQL 8 변환 위험 낮음	잘못된 날짜 3건 → 37,837건 → 115,653건	이관 절차 전면 수정
인덱스는 있을 것	DB 전체에 보조 인덱스 1개	인덱스 8개 설계
앱과 대시보드 API가 얽혀 있을 것	교집합 0건 (하루치 접근 로그)	“앱은 그대로, 대시보드만 새로”의 근거

조건이 빠져도 데이터가 새지 않게 만들었습니다

0건

침투 점검 8개 경로에서 새어 나간 데이터

문제

여러 병원이 한 시스템을
함께 씁니다. 조회마다
"내 병원 것만" 조건이 필요한데,
한 곳만 빠져도 무너집니다.

어떻게

요청이 들어오면 필터가 소속
병원 번호를 ScopedValue 에 묶고
그 안에서 처리합니다. 조건은
Hibernate 필터가 자동으로 붙입니다.

왜 ScopedValue 인가

가상 스레드라 ThreadLocal 은
지우는 걸 잊으면 다음 요청에
남습니다. ScopedValue 는 범위를
벗어나면 저절로 풀립니다.

결과

침투 점검 8개 경로 전부 차단
응답 108명분에서 가려지지
않은 이름 0건
범위 밖 조회는 404로 응답





AethraCorp

여러 고객 프로젝트가 올라가는 공통 인프라

2026.06 ~ 운영 중

개발자 2명 공동 구축 · 운영 (무보수 참여)

왜 만들었나

SI 회사라 고객 프로젝트가 계속 새로 생깁니다. 그때마다 서버에 들어가

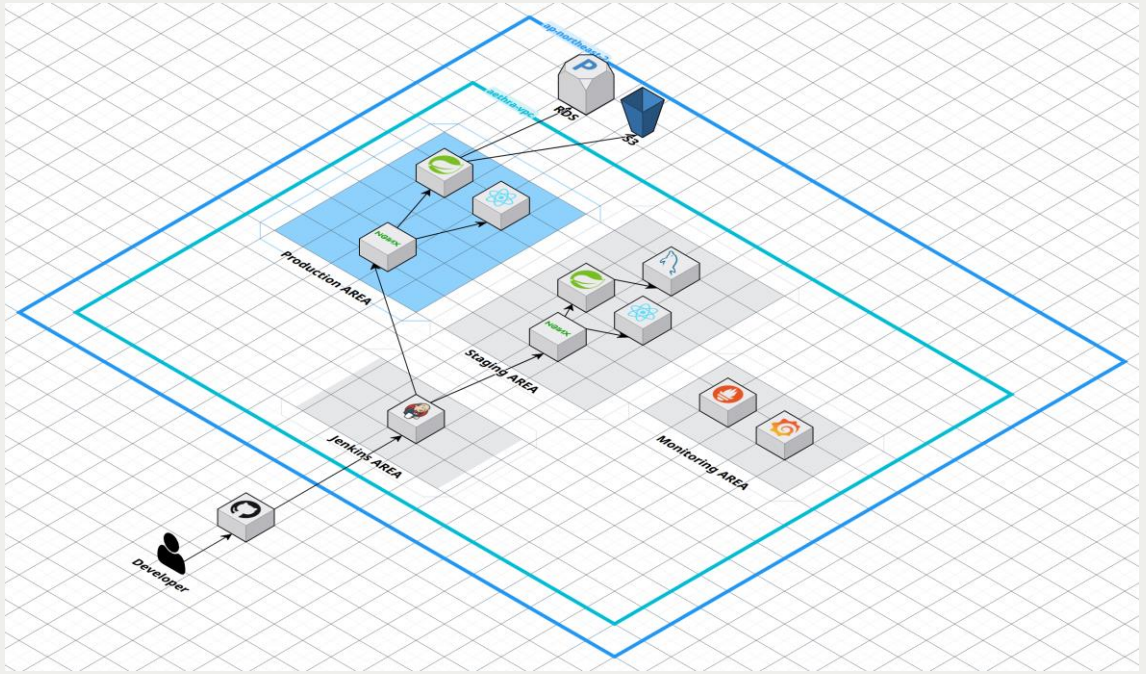
포트를 고르고, DB를 만들고, Nginx 설정과 인증서를 손으로 작업했습니다.

반복도 문제였지만, 포트를 겹치게 하면 기존 프로젝트가 죽습니다.

주요 기능

EC2 4대 (Jenkins / 스테이징 / 프로덕션 / 모니터링)

프로덕션 DB는 RDS, 스테이징은 EC2 내 MySQL (비용)



기술 스택

AWS EC2 · RDS · S3

Docker · Jenkins · Nginx · GHCR

Prometheus · Grafana · Loki

내 기여

CI/CD 파이프라인 · 온보딩 자동화

모니터링 · 로그 · 운영(DB 업그레이드, 디스크)

고객 검수용 피드백 위젯 단독 개발

결과 · 성과

온보딩 30~40분 → 약 5분

RDS 업그레이드 중단 2분, 코드 변경 0건

디스크 21GB 회수

서버 작업을 사람이 하지 않도록 옮겼습니다

5분

온보딩 소요 · 이전 30~40분

문제

새 프로젝트마다 같은 일을 반복했습니다. 포트를 직접 고르고, DB를 만들고, .env를 손으로 쓰고, Nginx와 인증서를 설정했습니다.

해결

템플릿 레포와 스크립트, Jenkins 파이프라인으로 묶었습니다. 포트는 서버의 레지스트리가 할당하고, 재배포해도 바뀌지 않습니다.

판단

모니터링 등록만은 일부러 자동화하지 않았습니다. 병합이 항상 실제 런칭은 아니고, 등록 실패가 배포를 막아서는 안 되기 때문입니다.



자동화의 목적은 시간 절약보다 손으로 할 때 틀릴 수 있는 것을 틀리지 않게 만드는 것이었습니다.

초록불이 거짓말을 하고 있었습니다

0건

가드 추가 이후 같은 사고

문제

어느 프로젝트가 오타 섞인 도메인에 통째로 배포됐습니다.

그러나 빌드는 정상으로 완료되었습니다.

원인

이름이 레포명 · Jenkins job명 · 스크립트 인자

세 곳에 따로 적혀 있었고, 어긋나도 검사하지

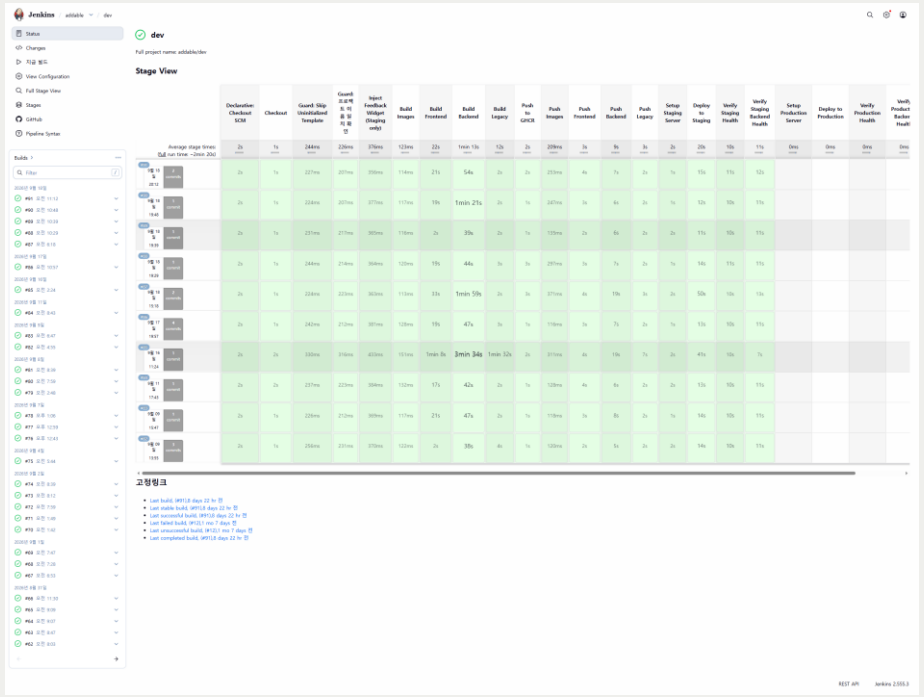
않았습니다. 확인 대상이 방금 만들어진 그 주소라

헬스체크까지 통과했습니다.

해결

세 이름이 일치하는지 검사하는 단계를 추가했습니다.

이미지 빌드보다 앞에 둔 것이 중요했습니다.



첫 빌드에서 포트 · DB · 인증서가 이미 만들어집니다. 검증이 자기 산출물을 대상으로 삼으면 검증이 되지 않는다고 판단했습니다.

AWS 사전 검사가 통과시킨 문제를 먼저 찾았습니다

2분

실제 중단 시간 · 코드 변경 0건

발단 RDS MySQL 8.0 지원 종료. 손대지 않으면

AWS가 정해진 시점에 업그레이드를 진행

발견 프로덕션 DB 계정이 구 방식 인증 사용

8.4에서 기본 비활성 · 사전 검사는 확인하지 않음

검증 임시 계정으로 확인 후

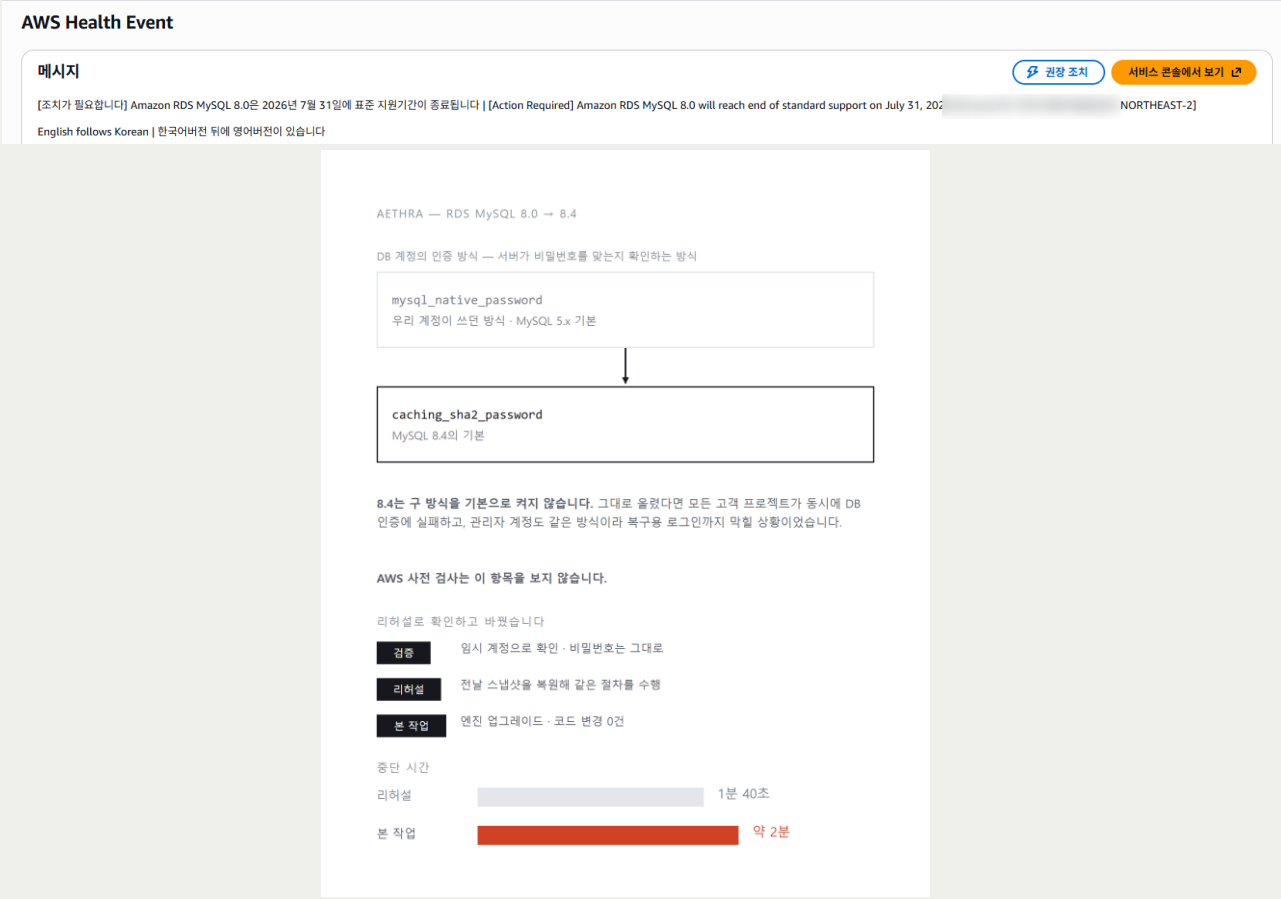
비밀번호는 유지하고 인증 방식만 전환

리허설 전날 스냅샷을 복원해 같은 절차 수행

복원본 기준 중단 1분 40초 실측

본 작업 수동 스냅샷 후 엔진 업그레이드

실제 중단 약 2분, 코드 변경 없음



대시보드 에러율이 서비스 상태와 달랐습니다

3 갈래

실사용자 · 크롤러 · 스캐너로 분리 집계

문제

에러율 지표가 높게 잡혔습니다. 서비스에 그 정도 문제가 있었다면 고객 문의가 먼저 도착했을 상황이었습니다.

원인

워드프레스 · PHP 취약점을 노린 스캐너가 존재하지 않는 경로를 긁고 다니며 404를 만들고 있었습니다. (/.env, /credentials.txt …)

해결

트래픽을 실사용자 · 크롤러 · 스캐너로 나눠 집계하도록 바꿨습니다. 하나의 숫자로 합치지 않고 구분한 채로 읽습니다.



봇 판별은 User-Agent와 요청 경로에 근거한 추정입니다. 그래서 합산 지표 하나로 결론 내리지 않고, 갈래별로 나눠 확인하도록 남겨 두었습니다.

우동

동아리 운영 통합 플랫폼

2025.08.25 ~ 09.29 · 6인

SSAFY 13기 특화 프로젝트 우수상 (2등)

왜 만들었나

팀원이 실제 동아리 회장을 하며 겪은 문제에서 출발했습니다.

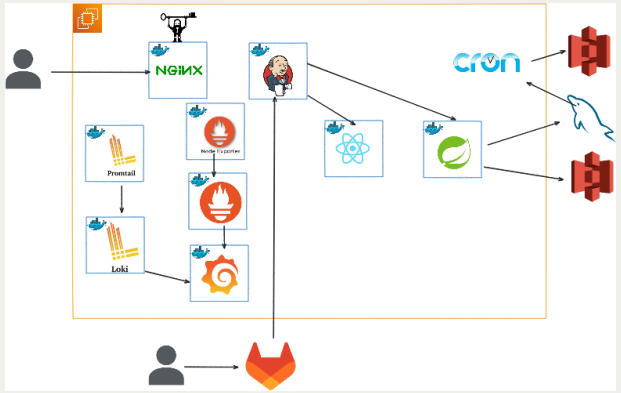
회비 미납자에게 일일이 연락하고, 회계를 엑셀로 정리하고,

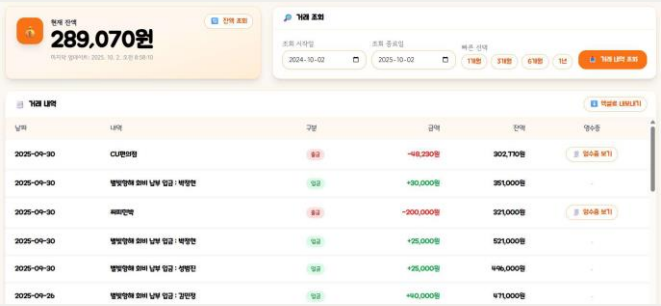
투표는 단톡방에서 했습니다. 이걸 한 곳에서 처리하게 만들자는 목표였습니다.

주요 기능

회비 납부 · 정산 / 공유 캘린더 · 채팅 / 투표

포인트 · 상점 / AI 마스코트 · MT 일정 생성





기술 스택

Java · Spring Boot · JPA · MySQL

Docker · Jenkins · Nginx · AWS EC2

Prometheus · Grafana · Loki · k6

내 기여

인프라 전담 — CI/CD, 컨테이너 구성,

모니터링 · 로그, DB 자동 백업

백엔드 — 알림 100% · 투표 95% · 공통코드 80%

결과 · 성과

기능 11개 · API 90개 규모

변경된 쪽만 빌드 · 배포하는 파이프라인

특화 프로젝트 우수상 (2등)

확인과 저장 사이를 막지 않으면 중복이 쌓입니다

1 행

동시 요청 200건에서 저장 · 이전 3행

문제

투표 참여는 "이미 참여했는지
확인 → 저장" 순서인데,
그 사이를 막는 장치가
없었습니다.

측정

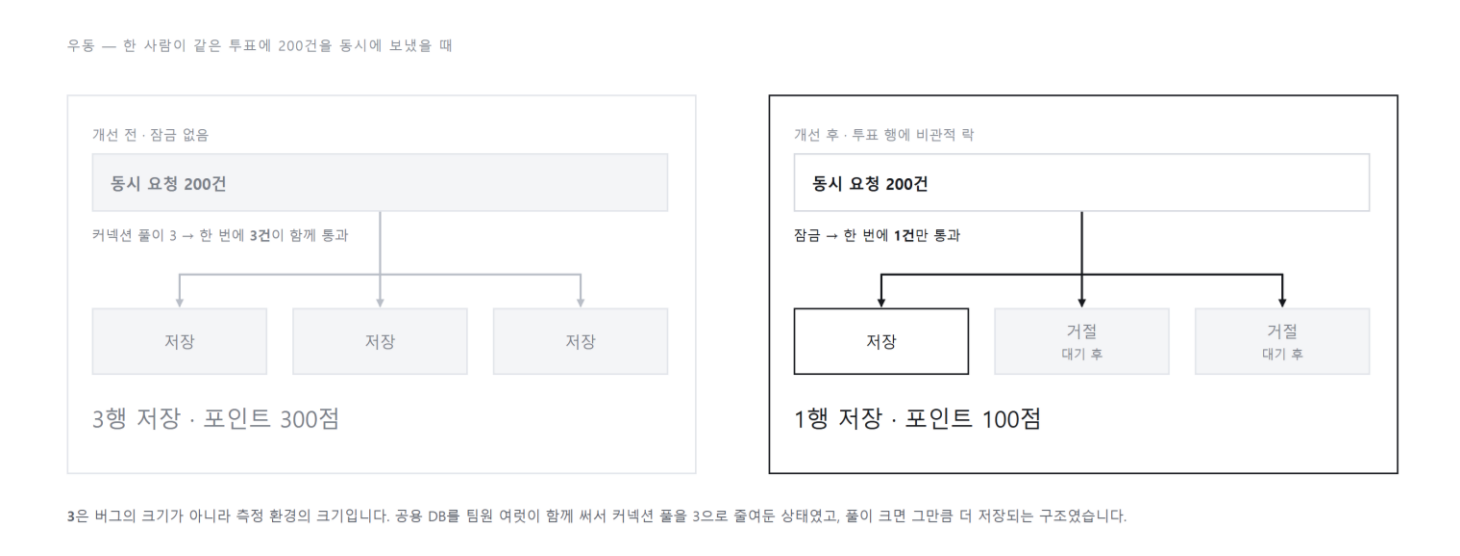
같은 사용자가 같은 투표에
200건을 동시에 보내고,
DB에 저장된 행 수로 확인
→ 3행이 들어가 있었습니다

해결

투표 행에 비관적 락을 걸어
확인부터 저장까지 하나씩
유니크 제약은 복수 선택
투표가 깨져서 못 씁니다

결과

중복 저장 3행 → 1행
포인트 300점 → 100점
응답 59ms → 488ms는
정합성을 위해 감수



투표가 늘어나는 만큼 쿼리도 같이 늘고 있었습니다

5건

투표 수와 무관하게 고정 · 이전 3N + 2

상황

투표 목록 화면입니다.

카드 한 장에 작성자 · 내 참여

여부 · 참여자 수 · 선택지 수가

같이 붙습니다.

문제

상세 조회는 쿼리 한 번으로

끝나게 해놓고, 목록은 그 값을

투표마다 따로 조회했습니다.

투표가 N개면 3N + 2건

해결

집계를 별도 쿼리로 묶어

투표 개수와 상관없이

고정 횟수로 끝나게 했습니다.

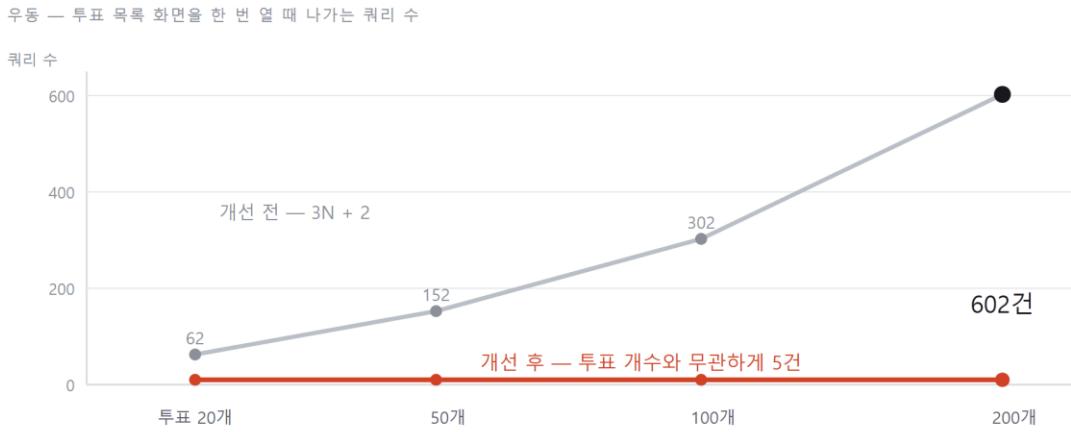
결과

투표 20개 62건 → 5건

투표 200개 602건 → 5건

개수가 늘어도 5건 고정





HAND

갤럭시워치 생체 데이터 기반 심리 모니터링

2025.10.10 ~ 11.20 · 7인

SSAFY 13기 자율 프로젝트 우수상 (2등)

왜 만들었나

소방관 · 경찰처럼 직무 스트레스가 큰 직군은 현장에 다시 투입되느라

마음을 돌볼 틈이 없습니다. 워치로 스트레스를 상시 측정하고,

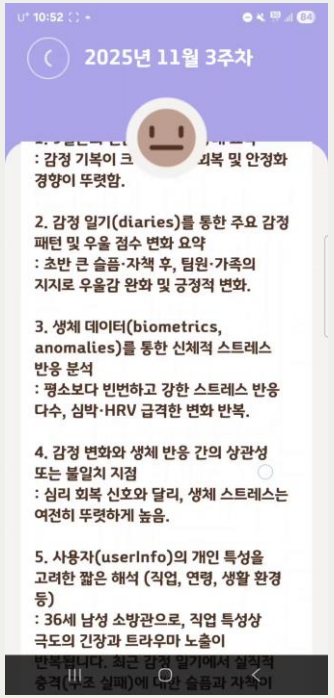
이상 신호가 보이면 그 자리에서 완화법을 제안하자는 목표였습니다.

주요 기능

워치 스트레스 측정 · 이상치 탐지 / 완화법 실행

음성 감정 다이어리 / 주간 · 월간 AI 보고서 / 관리자 대시보드





기술 스택

Java 25 · Spring Boot · JPA

MySQL · MongoDB · RabbitMQ

FCM · k6 · Docker

내 기여

백엔드 — 스트레스 산출 규칙 설계(WESAD 기준)

개인 기준값 API · 다이어리 AI 연동

하루 점수 · 주간/월간 보고서 · 알림 · 관리자 API

결과 · 성과

개인 기준값 3일 학습 후 개인화 판정

수집 API 응답 15.93ms → 3.86ms

자율 프로젝트 우수상 (2등)

워치는 저장 완료를 기다릴 필요가 없었습니다

3.86 ms

수집 API 평균 응답 · 이전 15.93ms

문제

워치는 10초마다 데이터를 보냅니다.

사용자 1,000명이면 초당 100건.

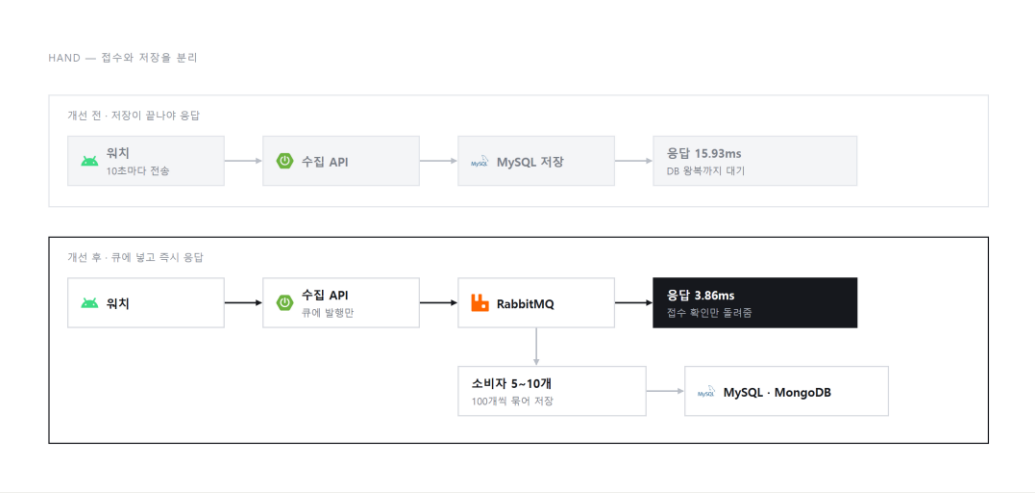
서버는 DB 저장이 끝난 뒤에야 응답했습니다.

해결

메시지 큐를 사이에 뒀습니다. API는 큐에 넣고 바로 응답하고, 저장은 소비자가 100개씩 묶어 처리합니다.

선택 이유

Kafka는 이 규모에 과했고, Redis Pub/Sub은 재시도가 어렵고, DB 큐 테이블은 줄이려던 DB 부하를 다시 늘립니다.



k6 — 초당 100건 · 60초 · 총 6,001건 · 동일 조건 전후 비교

지표	동기 저장	RabbitMQ 비동기	
평균 응답	15.93 ms	**3.86 ms**	↓ 75.8%
중앙값	15.23 ms	**3.76 ms**	
p90	19.87 ms	**5.24 ms**	
p95	22.27 ms	**5.71 ms**	↓ 74.4%
최대	67.61 ms	**38.05 ms**	
성공률	100%	100%	

줄어든 것은 **워치가 응답을 받기까지의 시간**이다.

줄어든 것은 워치가 응답을 받기까지의 시간입니다. 워치가 기다리지 않도록 만든 개선입니다.

ETCH

IT 취업 준비생을 위한 통합 취업 정보 플랫폼

2025.07 ~ 2025.08 · 6인

SSAFY 13기 공통 프로젝트

왜 만들었나

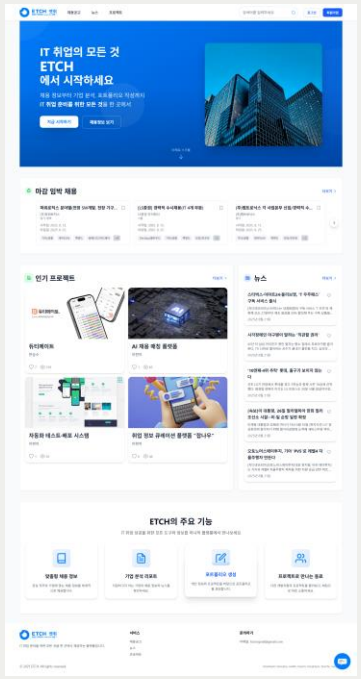
채용공고 · 기업 정보 · 뉴스 · 자소서 · 포트폴리오가 전부 다른 곳에 흩어져 있습니다. 한 곳에서 보게 만들자는 목표였습니다.

저는 이 프로젝트에서 처음으로 프론트엔드를 맡았습니다.

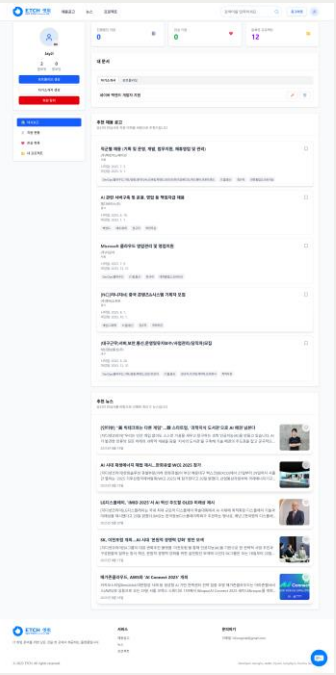
주요 기능

채용공고 · 기업 · 뉴스 통합 / 맞춤 추천 / 통합 검색

자소서 · 포트폴리오 작성 / 프로젝트 SNS







기술 스택

React 19 · TypeScript · Vite

TailwindCSS · Zustand

React Router · FullCalendar

내 기여

프론트엔드 (2인 중 1인, 담당 화면 기준)

로그인 · OAuth / 채용정보 리스트 · 캘린더 · 필터

마이페이지 전체 / 통합검색 / 토큰 자동 갱신

결과 · 성과

LCP 18.9초 → 1.3초

랜딩 이미지 20.71MB → 2.97MB

Lighthouse 성능 59 → 78

첫 화면 로딩을 18.9초에서 1.3초로 줄였습니다

1.3 초

첫 화면 로딩 LCP · 이전 18.9초

측정

첫 화면이 뜨는 데 18.9초

추정 대신 번들 분석기부터 붙였더니

랜딩 이미지가 전체 20.71MB로 압도적

손본 것

해상도를 표시 크기에 맞춰 재생성

WebP로 변환

첫 화면 이미지만 우선 로딩

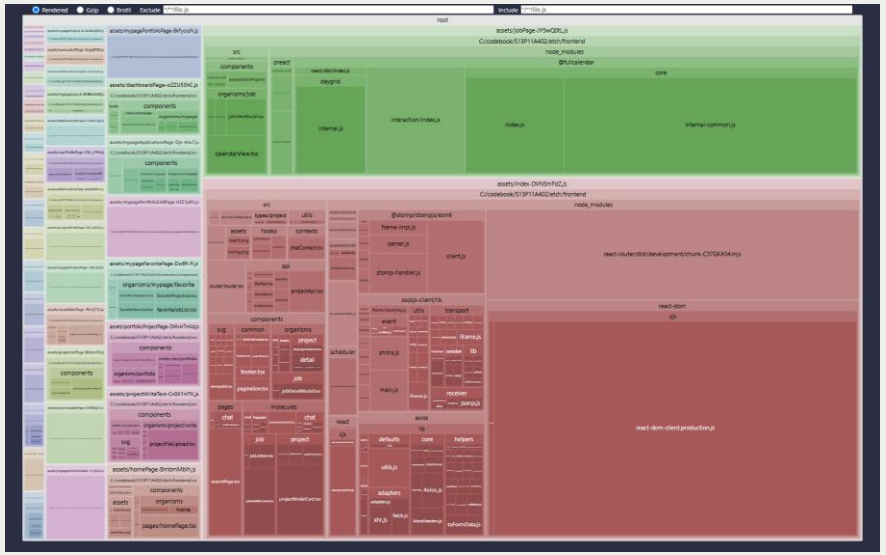
가로세로 지정 · 번들 분할

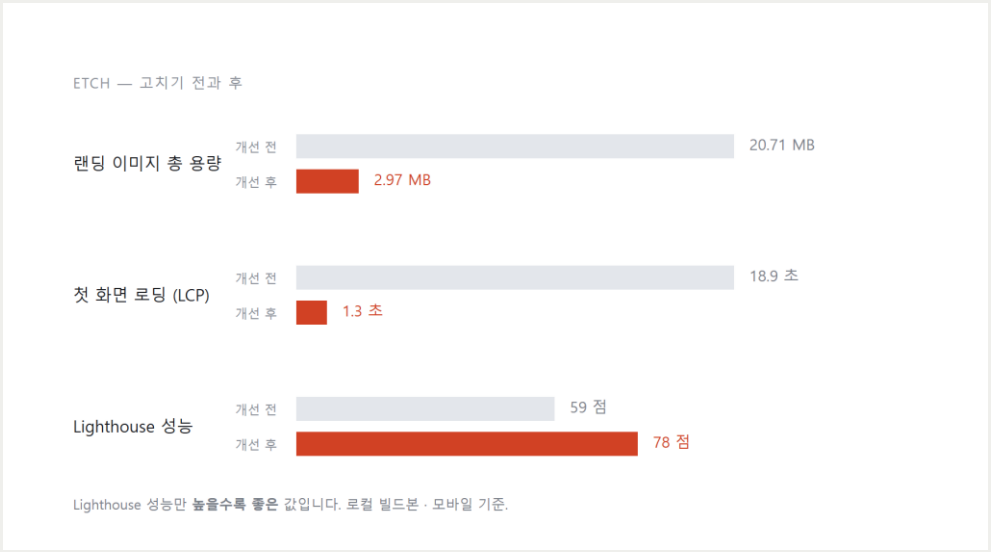
결과

랜딩 이미지 20.71MB → 2.97MB

첫 화면 로딩 18.9초 → 1.3초

Lighthouse 성능 59 → 78



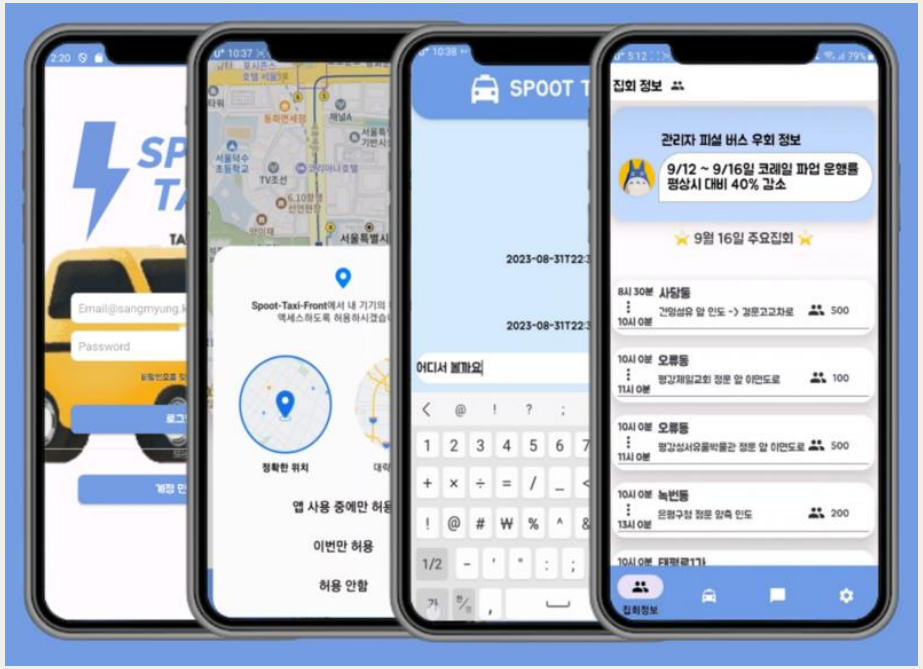


SpootTaxi

2023.05 ~ 10 · 2인

대학 캡스톤 · 서버와 앱 전부 담당

등교 택시 동승 매칭 앱 · 첫 백엔드 프로젝트





왜 만들었나

학교 앞 버스가 시위로 우회하거나 멈추는 일이 잦아 택시를 탈 수밖에 없었습니다. 학생들은 에브리타임에 글을 올리고, 댓글을 기다리고, 오픈채팅방을 새로 파는 5단계를 거치고 있었습니다.

주요 기능

위치 기반 동승자 매칭 / 앱 내 실시간 채팅

집회 · 우회 정보 / 매칭 성공 푸시 알림

기술 스택

Java · Spring Boot · JPA · QueryDSL

MySQL · WebSocket(STOMP) · Kafka

Android(Java) · FCM

내 기여

채팅 전 구간 — DB 설계 · API · 소켓 서버 분리

Kafka 연동 · 안드로이드 채팅 화면

집회 정보 API · 회원 API · 조회 쿼리 최적화

결과 · 성과

소켓 서버 분리로 채팅과 API 부하 분리

졸업작품 전시회에서 학생 체험 진행

나눈 뒤에도 화면이 API 서버에 묶여 있었습니다

2 갈래

보이는 경로와 저장 경로 분리

초기 구조

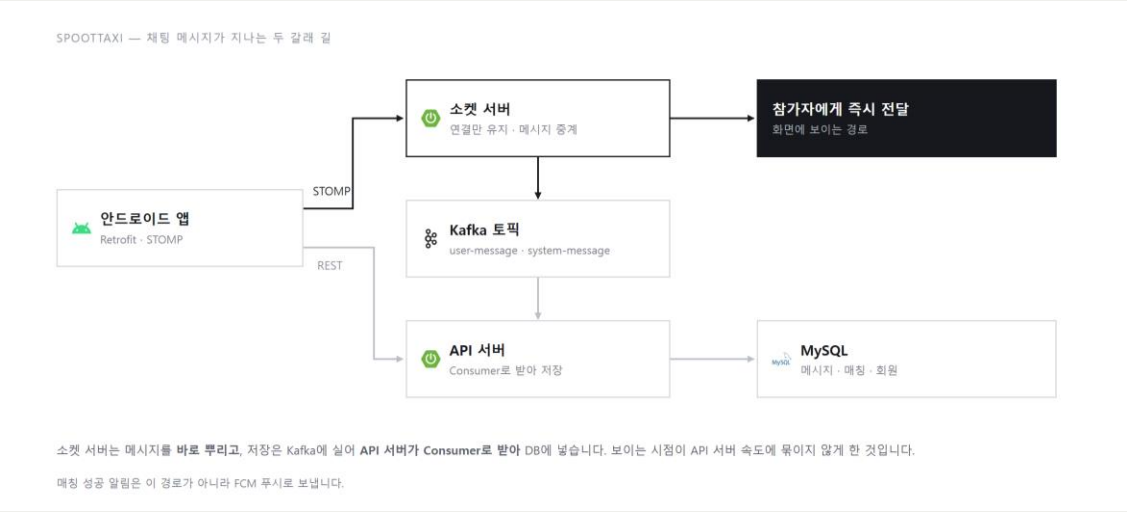
채팅 연결은 앱을 켜둔 내내 유지되고, API 요청은 받고 응답하면 끝납니다. 성격이 다르니 소켓 서버를 분리했습니다.

문제

소켓 서버가 API 서버에 저장을 요청하고 그 응답을 받은 뒤에 뿌렸습니다. 화면에 보이는 시점이 API 서버 속도에 묶여 있었습니다.

해결

메시지 브로커를 사이에 뒀습니다. 바뀐 것은 순서가 아니라 의존성입니다. 보이는 것과 저장되는 것이 서로를 기다리지 않게 됐습니다.



읽어주셔서 감사합니다

brian_als@naver.com

github.com/ks0927